

Sessions 4 & 5: Processes

1. What is a Process?

A **process** is an instance of a running program, including the program code and its current activity. A process requires system resources such as CPU time, memory, and I/O devices.

- **Preemptive Processes:** Processes that can be forcibly interrupted by the OS to allow another process to execute (e.g., multitasking OS).
 - **Non-preemptive Processes:** Processes that run until completion without being interrupted (e.g., older single-tasking systems).
-

2. Difference Between Process and Thread

Feature	Process	Thread
Definition	Independent execution unit with its own resources.	Lightweight sub-unit within a process.
Resource Allocation	Has its own memory space and system resources.	Shares memory and resources with its process.
Communication	Inter-process Communication (IPC) mechanisms are required.	Threads within a process can directly communicate.
Overhead	High (due to context switching).	Low (threads share context).
Example	Running multiple programs like a browser and editor.	Executing multiple tabs within a browser.

3. Process Management and Life Cycle

Process management involves the creation, scheduling, and termination of processes.

Process Life Cycle States:

1. **New:** The process is being created.
 2. **Ready:** The process is waiting for CPU allocation.
 3. **Running:** The process is actively executing.
 4. **Blocked:** The process is waiting for an event (e.g., I/O).
 5. **Terminated:** The process has completed execution.
-

4. Schedulers

Schedulers decide which process should execute next.

- **Short-term Scheduler:**
 - Responsible for selecting processes from the ready queue for execution on the CPU.
 - Example: CPU scheduling.
- **Medium-term Scheduler:**
 - Temporarily removes processes from memory to reduce the load (swapping).
 - Example: Managing background applications.

- **Long-term Scheduler:**
 - Determines which processes are admitted to the system for processing.
 - Example: Controlling the mix of I/O-bound and CPU-bound processes.
-

5. Process Scheduling Algorithms

Scheduling algorithms manage how processes are selected for execution. Key metrics include **turnaround time**, **waiting time**, and **response time**.

5.1 First Come, First Serve (FCFS)

- Executes processes in the order they arrive.
 - **Advantages:** Simple to implement.
 - **Disadvantages:** May cause long waiting times.
 - **Example:**
 - Process Arrival: P1(0), P2(1), P3(2)
 - Burst Times: P1=5, P2=3, P3=1
 - Execution Order: P1 → P2 → P3
 - Turnaround Time = Completion Time - Arrival Time.
-

5.2 Shortest Job First (SJF)

- Executes the process with the shortest burst time first.
 - **Advantages:** Minimizes average waiting time.
 - **Disadvantages:** Starvation of long processes.
 - **Example:**
 - Burst Times: P1=6, P2=8, P3=7, P4=3
 - Execution Order: P4 → P1 → P3 → P2.
-

5.3 Priority Scheduling

- Executes processes based on priority (higher priority executes first).
 - **Example:**
 - Priorities: P1=2, P2=1, P3=3
 - Execution Order: P2 → P1 → P3.
-

5.4 Round Robin (RR)

- Each process gets a fixed time slice (quantum).
- **Advantages:** Ensures fairness.
- **Disadvantages:** May have higher turnaround times.
- **Example:**
 - Time Quantum = 3ms.
 - Burst Times: P1=5, P2=7, P3=3

- Execution: P1(3ms) → P2(3ms) → P3(3ms) → P1(remaining 2ms) → P2(remaining 4ms).
-

5.5 Multilevel Queue Scheduling

- Divides processes into multiple queues based on characteristics like priority or process type.
 - Example:
 - Foreground (interactive) and background (batch) queues.
-

6. Belady's Anomaly

- **Definition:** In some cases, increasing the number of page frames in memory increases the number of page faults.
 - **Example:** Occurs in FIFO page replacement algorithms.
-

7. Examples Associated with Scheduling Algorithms

- Example for **Turnaround Time**:
 - Turnaround Time = Completion Time - Arrival Time.
 - Processes: P1(Arrival=0, Burst=4), P2(Arrival=1, Burst=3), P3(Arrival=2, Burst=1)
 - Using FCFS:
 - P1 completes at 4, P2 completes at 7, P3 completes at 8.
 - Turnaround Times: P1=4, P2=6, P3=6.
-

8. Process Creation Using fork, waitpid, and exec

8.1 fork() System Call

- Creates a new process (child) by duplicating the parent process.
- **Example:**

```
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child process\n");
    } else {
        printf("Parent process\n");
    }
    return 0;
}
```

8.2 waitpid() System Call

- Makes the parent wait for a specific child process to terminate.
- **Example:**

```
pid_t pid = fork();
if (pid > 0) {
    waitpid(pid, NULL, 0);
    printf("Child process terminated\n");
}
```

8.3 exec() System Call

- Replaces the current process image with a new program.
- **Example:**

```
#include <unistd.h>
int main() {
    char *args[] = {"ls", "-l", NULL};
    execvp("ls", args);
    return 0;
}
```

9. Parent and Child Processes

- **Parent Process:** The process that initiates a new process using `fork()`.
 - **Child Process:** The newly created process.
 - **Example:**
 - In the `fork()` example above, the `if` block differentiates parent and child.
-

10. Orphan and Zombie Processes

- **Orphan Process:**
 - A child process whose parent has terminated.
 - Adopted by `init` process (PID 1).
- **Zombie Process:**
 - A process that has terminated but still has an entry in the process table because the parent hasn't read its exit status.
- **Example:**

```
#include <unistd.h>
#include <sys/types.h>
#include <stdio.h>
int main() {
    if (fork() > 0) {
        sleep(10); // Parent waits
    } else {
        printf("Child process exiting\n");
        _exit(0); // Exit without cleanup
    }
    return 0;
}
```

This detailed explanation provides a complete overview of processes, their management, scheduling algorithms, and process-related system calls.